

```

import path from 'node:path';
import { fileURLToPath } from 'node:url';
import * as db from './db.js';
import { loadPackages } from './kernel/registry.js';
import './kernel/canonical.js'; // registers kernel capabilities
import './kernel/channels.js'; // registers the android route capabilities
import { installFanOut } from './kernel/fanout.js';
import { bind } from './kernel/providers.js';

const here = path.dirname(fileURLToPath(import.meta.url));
export const ROOT = path.resolve(here, '..');

export async function boot({ url } = {}) {
  await db.connect({ url, schemaDir: path.join(ROOT, 'db') });
  const packages = await loadPackages(path.join(ROOT, 'modules'));
  installFanOut();
  await defaults();
  return { packages };
}

// A context is who is acting, on behalf of which business.
export async function contextFor({ personId, businessSlug }) {
  const business = await db.one(`select * from business where slug = $1`, [businessSlug]);
  if (!business) throw new Error(`no business "${businessSlug}"`);
  const person = personId ? await db.one(`select * from person where id = $1`, [personId]) : null;
  const membership = person
    ? await db.one(`select * from membership where person_id = $1 and business_id = $2`, [person.id, business.id])
    : null;
  return { businessId: business.id, business, person, membership };
}

// Platform defaults. A business overrides any of these with one row.
async function defaults() {
  const { bind: bindSlot } = await import('./kernel/providers.js');
  const pairs = [
    ['workspace.executor', 'android_cloud'],
    ['builder', 'composer'],
    ['harness', 'loop_harness'],
    ['model', 'hosted_models'],
  ];
  const { q } = await import('./db.js');
  for (const [slot, pkg] of pairs) {
    const existing = await q(

```

```
`select 1 from provider_binding where slot_key = $1 and business_id is null
[slot, pkg]
);
if (existing.length === 0) {
  await bindSlot({ businessId: null, slot, packageKey: pkg,
    config: slot === 'model' ? { simulate: true } : {} });
}
}
}
```